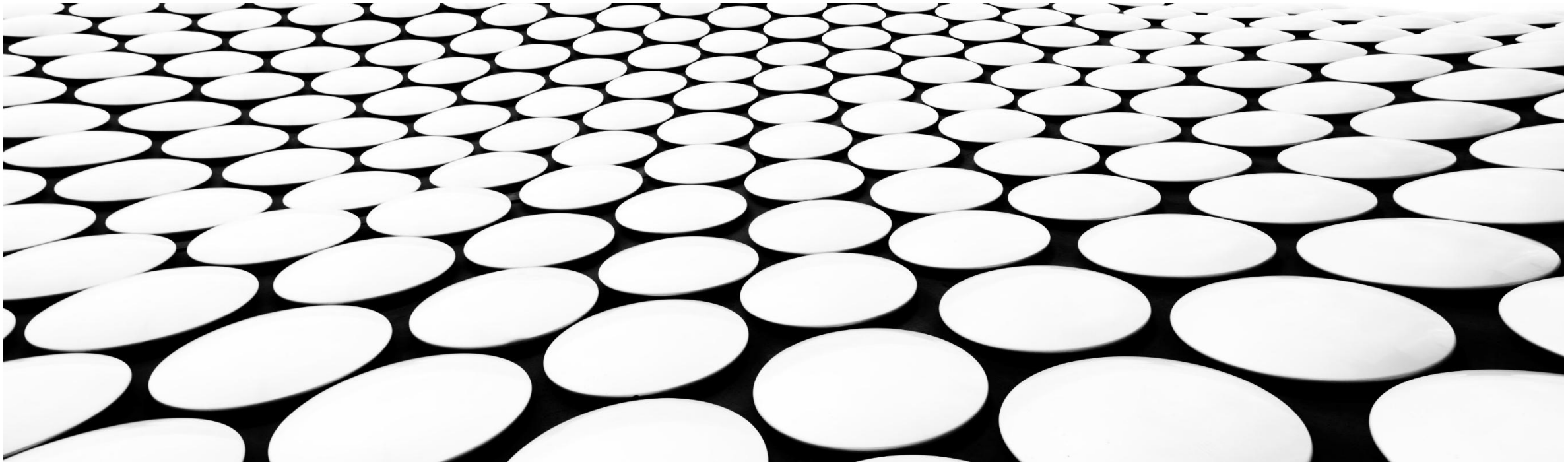
CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

LECTURE 20: **PROCESS SYNCHRONIZATION - I**

DR. ARIJIT ROY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA



BACKGROUND

- Processes can execute concurrently
  - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

Process type

Independent: Do not affect other processes

Co-operative: Affects other processes

Two processes P_i and P_j

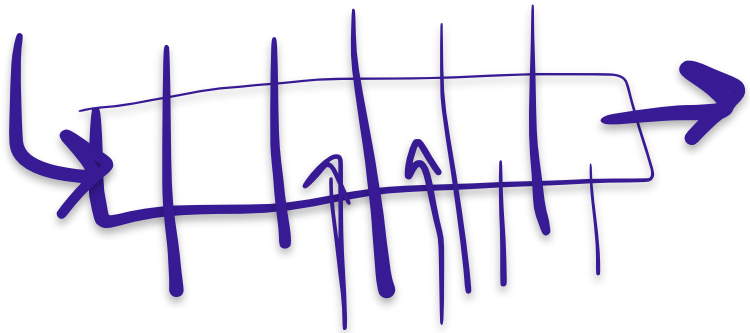
```
// For Process Pi
int a = Shared;
a++;
Sleep(1);
Shared = a;
```

```
// For Process Pj
int b = Shared;
b--;
Sleep(1);
Shared = b;
```

Race Condition

PRODUCER

Illustration of the problem:
Suppose that we wanted to provide a solution to the consumer-producer problem that fills ***all*** the buffers. We can do so by having an integer **counter** that keeps track of the number of full buffers. Initially, **counter** is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.



```
while (true) {
    /* produce an item in next produced */

    while (counter == BUFFER_SIZE)
    ;  /* do nothing */

    buffer[in] = next_produced;

    in = (in + 1) % BUFFER_SIZE;

    counter++;
}
```

CONSUMER

```
while (true) {
    while (counter == 0)
        ; /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
        counter--;
    /* consume the item in next consumed */
}
```

RACE CONDITION

- **`counter++`** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **`counter--`** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute **`register1 = counter`** {register1 = 5}
S1: producer execute **`register1 = register1 + 1`** {register1 = 6}
S2: consumer execute **`register2 = counter`** {register2 = 5}
S3: consumer execute **`register2 = register2 - 1`** {register2 = 4}
S4: producer execute **`counter = register1`** {counter = 6 }
S5: consumer execute **`counter = register2`** {counter = 4}

CRITICAL SECTION PROBLEM

- Consider system of ***n*** processes $\{p_0, p_1, \ldots p_{n-1}\}$
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section
- ***Critical section problem*** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

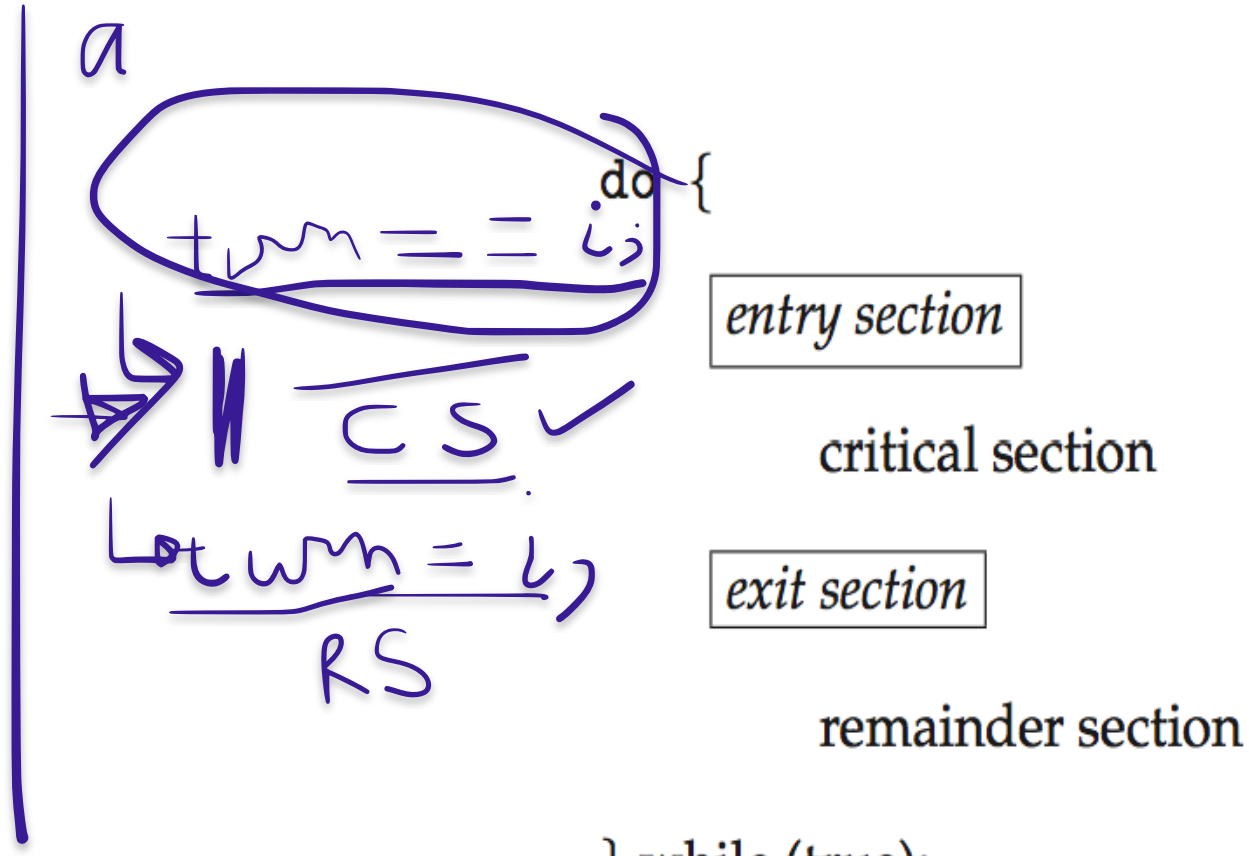
CRITICAL SECTION

- General structure of process P_i

```
do {
        entry section
            critical section
        exit section
                remainder section
} while (true);
```

ALGORITHM FOR PROCESS P_I

```
do {

    while (turn == j);

        critical section

    turn = j;

        remainder section

} while (true);
```



SOLUTION TO CRITICAL-SECTION PROBLEM

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning **relative speed** of the ***n*** processes

CRITICAL-SECTION HANDLING IN OS

Two approaches depending on if kernel is preemptive or non-preemptive

- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
  - Essentially free of race conditions in kernel mode

SOLUTION TO CRITICAL SECTION

CRITICAL-SECTION HANDLING IN OS

Two approaches depending on if kernel is preemptive or non-preemptive

- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
  - Essentially free of race conditions in kernel mode

SOLUTION TO CRITICAL-SECTION PROBLEM USING LOCKS

```
do {
    acquire lock
        critical section
    release lock
        remainder section
} while (TRUE);
```

MUTEX LOCKS

- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
- Protect a critical section by first **acquire()** a lock then **release()** the lock
  - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
  - Usually implemented via hardware atomic instructions
- But this solution requires **busy waiting**
  - This lock therefore called a **spinlock**

ACQUIRE() AND RELEASE()

```
acquire()
{
while (!available)
          ; /* busy wait */
available = false;
}
```

```
release()
{
available = true;
}
```

```
do {
    acquire lock
        critical section
    release lock
        remainder section
} while (true);
```

In a simplified way

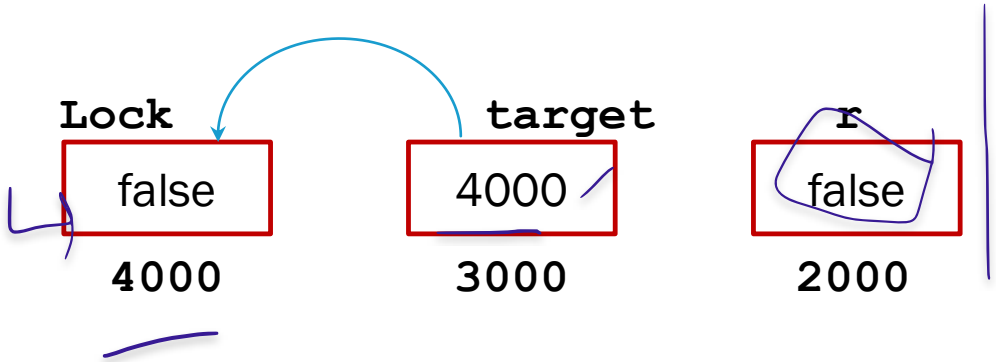
```
Step 1:  while(lock == 1);
Step 2:  Lock = 1;
Step 3:  CS
Step 4:  Lock = 0
```

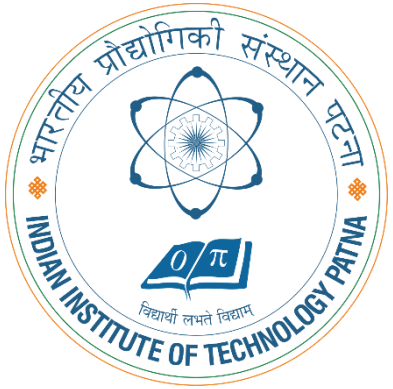
Does it ensure mutual exclusion?

TEST AND SET

```
acquire()
{
while(test_and_set(&Lock))
            ; /* busy wait */
}

Critical Section

Lock= false

}
```

```
boolean test_and_set(boolean *target)
{
        boolean r = *target;
        *target = true;
        return r;
}
```




भारतीय प्रौद्योगिकी संस्थान पटना
INDIAN INSTITUTE OF TECHNOLOGY PATNA
विद्यार्थी लभते विद्याम्

THANK YOU!